Отчёт по лабораторной работе №6

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-25 **Иванов Ярослав Дмитриевич** № по списку 6.

Контакты: xomeshok@gmail.com

Работа выполнена: «15» марта 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «17» марта 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. Тема: Программирование на интерпретируемых командных языках ОС UNIX.

2. Цель работы:

Разработать сценарий (скрипт) для командного интерпретатора Bash и программу на языке Python для кодирования по шифру Цезаря всех текстовых файлов в указанном каталоге и его поддиректориях с использованием утилиты tr.

3. Задание (Вариант № 6): Кодирование по шифру Цезаря всех текстовых файлов в указанном каталоге и его поддиректориях с использованием утилиты tr.

4. Оборудование: Оборудование ПЭВМ студента, если использовалось:

Процессор: Apple M1 @ 3.2GHz 3.2 GHz, ОП 8 ГБ, SSD 256 ГБ

5. Программное обеспечение: Программное обеспечение ЭВМ студента, если использовалось:

Операционная система семейства Unix, наименование macOS Tahoe, версия 26.0, интерпретатор команд bash версия 5.2.

Система программирования: Xcode Command Line Tools версия 15.4. Редактор текстов: VS Code версия 1.107.1.

Язык программирования: Bash, Python 3.

6. Идея, метод, алгоритм решения задачи [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:

Идея:

Шифр Цезаря заменяет каждую букву текста на букву, сдвинутую на K позиций вперёд по алфавиту. При достижении конца алфавит закольцовывается. Регистр букв сохраняется, прочие символы не изменяются. Для выполнения замены символов используется стандартная утилита tr, которая принимает два набора символов: исходный алфавит и сдвинутый алфавит — и заменяет каждый символ первого набора соответствующим символом второго набора.

Метод:

1. Получение аргументов: Скрипт принимает два параметра: путь к каталогу для обработки и ключ шифрования (целое число).

2. Построение алфавитов для tr: На основе ключа K вычисляется сдвиг (остаток от деления на 26), после чего формируются строки исходного и сдвинутого алфавитов для строчных и прописных букв.

3. Рекурсивный обход: Используя стандартные средства (команда find в Bash и функция os.walk() в Python), выполняется рекурсивный обход всех файлов с расширением .txt в указанном каталоге.

4. Шифрование каждого файла:

- Содержимое файла передаётся в утилиту tr через стандартный ввод.
- Результат записывается во временный файл.
- Временный файл атомарно заменяет оригинальный (mv в Bash, os.replace() в Python).

Алгоритм:

```
dir  = аргумент1
key  = аргумент2
shift = key mod 26
lower = "abcdefghijklmnopqrstuvwxyz"
upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
shifted_lower = lower[shift:] + lower[:shift]
shifted_upper = upper[shift:] + upper[:shift]
FROM = lower + upper
TO   = shifted_lower + shifted_upper
Для каждого file в рекурсивный_обход(dir, '*.txt'):
    tmp = создать_временный_файл()
    tr FROM TO < file > tmp
    mv tmp file
```

7. Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию).

План работы:

1. Анализ предметной области: изучить принцип работы шифра Цезаря и утилиты tr.

2. Проектирование алгоритма: определить способ построения сдвинутых алфавитных строк для tr по ключу.

3. Реализация на Bash:

- Использовать срезы строк ${str:N}${str:0:N} для построения сдвинутого алфавита.
- Использовать find для рекурсивного обхода.
- Использовать mktemp и mv для безопасной перезаписи файлов.

4. Реализация на Python:

- Использовать модули os, subprocess, argparse, tempfile.
- Применить os.walk() для обхода.
- Использовать subprocess.run(['tr', ...]) для шифрования.

5. Тестирование: проверка на различных структурах каталогов и файлах.

Тесты:

```
mkdir -p test_dir/sub1/sub2
echo 'hello world' > test_dir/a.txt
echo 'bash script' > test_dir/sub1/b.txt
echo 'python'      > test_dir/sub1/sub2/c.txt

./solution.sh -d ./test_dir -k 3
python3 solution.py -d ./test_dir -k 3

cat test_dir/a.txt           # ожидается: khoor zruog
cat test_dir/sub1/b.txt      # ожидается: edvk vfulsw
cat test_dir/sub1/sub2/c.txt # ожидается: sbwkrq

# Дешифрование обратным ключом
./solution.sh -d ./test_dir -k 23
cat test_dir/a.txt           # ожидается: hello world
```

Соображения по тестированию:

➢ Проверка на пустом каталоге: Скрипт не должен ничего изменять и завершаться без ошибок.

➢ Проверка с каталогом без .txt файлов: Скрипт должен завершаться без ошибок.

➢ Проверка с файлами в поддиректориях: Убедиться, что обход работает рекурсивно.

➢ Проверка дешифрования: Применение ключа (26 - K) должно восстанавливать исходный текст.

➢ Проверка граничных значений: K=0 (текст не меняется), K=26 (эквивалентно K=0), K=-3 (сдвиг влево).

Пункты 1-7 отчета составляются строго до начала лабораторной работы.

Допущен к выполнению работы. Подпись преподавателя _____

8. Распечатка протокола (подклеить листинг окончательного варианта программы с тестовыми примерами, подписанный преподавателем).

```
#!/bin/bash

show_help() {
    cat << 'EOF'
Usage: solution.sh [OPTION]... -d DIRECTORY -k KEY
Encode all text files in DIRECTORY and its subdirectories using Caesar
cipher.
Files are modified in-place. Only latin letters are shifted; other chars
unchanged.

Mandatory arguments to long options are mandatory for short options too.
  -d, --directory DIR    path to the target directory
  -k, --key KEY          shift value (integer, e.g. 3)
  -h, --help             display this help and exit

Examples:
  solution.sh -d ./texts -k 3
  solution.sh --directory /home/user/docs --key 13
EOF
}
```

```
make_tr_args() {
    local shift=$1
    shift=$(( shift % 26 ))
    if [ "$shift" -lt 0 ]; then
        shift=$(( shift + 26 ))
    fi
    local lower="abcdefghijklmnopqrstuvwxyz"
    local upper="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    local shifted_lower="${lower:$shift}${lower:0:$shift}"
    local shifted_upper="${upper:$shift}${upper:0:$shift}"
    TR_FROM="${lower}${upper}"
    TR_TO="${shifted_lower}${shifted_upper}"
}

DIRECTORY=""
KEY=""

while [[ $# -gt 0 ]]; do
    case "$1" in
        -h|--help)
            show_help
            exit 0
            ;;
        -d|--directory)
            DIRECTORY="$2"
            shift 2
            ;;
        -k|--key)
            KEY="$2"
            shift 2
            ;;
        *)
            echo "Unknown option: $1"
            show_help
            exit 1
            ;;
    esac
done

if [ -z "$DIRECTORY" ] || [ -z "$KEY" ]; then
    show_help
    exit 1
fi

if ! [[ "$KEY" =~ ^-?[0-9]+$ ]]; then
    echo "Error: KEY must be an integer."
    show_help
    exit 1
fi

if [ ! -d "$DIRECTORY" ]; then
    echo "Error: directory '$DIRECTORY' does not exist."
    exit 1
fi

make_tr_args "$KEY"

find "$DIRECTORY" -type f -name "*.txt" | while IFS= read -r file; do
    echo "Encoding: $file"
    local_tmp=$(mktemp)
    tr "$TR_FROM" "$TR_TO" < "$file" > "$local_tmp"
```

```
    mv "$local_tmp" "$file"
done

echo "Done. All .txt files in '$DIRECTORY' have been encoded with key=$KEY."
///
#!/usr/bin/env python3

import argparse
import os
import subprocess
import tempfile


def make_tr_args(shift: int) -> tuple[str, str]:
    shift = shift % 26
    lower = "abcdefghijklmnopqrstuvwxyz"
    upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    shifted_lower = lower[shift:] + lower[:shift]
    shifted_upper = upper[shift:] + upper[:shift]
    return lower + upper, shifted_lower + shifted_upper


def encode_file(filepath: str, tr_from: str, tr_to: str) -> None:
    print(f"Encoding: {filepath}")
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        tmp_path = tmp.name
    with open(filepath, "rb") as src, open(tmp_path, "wb") as dst:
        subprocess.run(["tr", tr_from, tr_to], stdin=src, stdout=dst,
check=True)
    os.replace(tmp_path, filepath)


def process_directory(directory: str, tr_from: str, tr_to: str) -> None:
    for root, dirs, files in os.walk(directory):
        for filename in files:
            if filename.endswith(".txt"):
                filepath = os.path.join(root, filename)
                encode_file(filepath, tr_from, tr_to)


def main():
    parser = argparse.ArgumentParser(
        prog="solution.py",
        description="Encode all text files in DIRECTORY and its
subdirectories",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=("Examples:\n"
                "  solution.py -d ./texts -k 3\n"
                "  solution.py --directory /home/user/docs --key 13")
    )
    parser.add_argument("-d", "--directory", required=True, metavar="DIR",
                        help="path to the target directory")
    parser.add_argument("-k", "--key", required=True, type=int,
metavar="KEY",
                        help="shift value (integer, e.g. 3)")
    args = parser.parse_args()
    if not os.path.isdir(args.directory):
        parser.error(f"Directory '{args.directory}' does not exist.")
    tr_from, tr_to = make_tr_args(args.key)
    process_directory(args.directory, tr_from, tr_to)
    print(f"Done. All .txt files in '{args.directory}' encoded with
key={args.key}.")
```

```
if __name__ == "__main__":
    main()
```

9. Дневник отладки должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. Замечания автора по существу работы: замечания отсутствуют.

11. Выводы:

В ходе выполнения лабораторной работы были разработаны два скрипта (на Bash и Python) для кодирования по шифру Цезаря всех текстовых файлов в указанном каталоге и его поддиректориях с использованием утилиты tr. Алгоритм работы основан на динамическом построении сдвинутого алфавита по ключу и передаче его в утилиту tr. Применение временного файла при перезаписи исключает потерю данных. Оба скрипта имеют идентичный интерфейс командной строки и успешно протестированы в среде macOS Tahoe. Работа позволила закрепить навыки написания скриптов на Bash и Python, работы с аргументами командной строки, рекурсивного обхода файловой системы и применения стандартных утилит UNIX.

Подпись студента _____